

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$. Siehe **DMI**.

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$
$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$$\neg \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} (\neg P_i)$$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff	Bedeutung
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\bar{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

n -Tupel:

$$\prod_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

Begriff	Bedeutung
Leeres Wort	Die Zeichenkette $\epsilon \in \Sigma^0 = \{\epsilon\}$ der Länge 0 heisst das leere Wort .
Menge aller Wörter	Leeres Wort ist immer drin $\Sigma^* = \{\epsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$
Abgekürzte Schreibweisen von Wörtern	$(A, u, t, o, S, p, r) = AutoSpr$ $a^3b^5 = aaabbbbbb$ $b^5a^3 = bbbbaaaa$
Sprache	Teilmenge $L \subset \Sigma^*$

WORTLÄNGE

Begriff	Bedeutung
Länge des Wortes	$w \in \Sigma^n \Rightarrow w = n$, n ist Länge des Wortes w
Anzahl Zeichen	Sei $w \in \Sigma^*$, $a \in \Sigma$. Dann ist $ w _a$ die Anzahl Zeichen a im Wort w

AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist aufeinanderfolgend zweimal ein Ziffernsystem}\}$ akzeptiert

θ	1	2
g	g	u
u	u	g

PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- $|xy| \leq N$
- $|y| > 0$
- $xy^kz \in L \forall k \in \mathbb{N}$

Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^kz \in L$).

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- Gemäss Pumping Lemma gibt es die Pumping Length N
- Wähle ein Wort $w \in L$ mit $|w| \geq N$
- Aufteilung des Wortes gemäss Pumping Lemma
- Auswirkung des Pumpens
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$

Startzustand: $q_0 \in Q$

Akzeptierzustände: $F \subset Q$

Faustregeln

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei } 0\}$

DEA-Lösung	NEA-Lösung

UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher automaten, PEa)

THOMPSON-NEA

- Zustand = Menge der möglichen Zustände
- Akzeptieren = Ob NEA im Zustand sein könnte

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	Anwendungsfall
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitsaussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

	Kontextfrei	Regulär	CFG	PDA
DEA				
NEA				
regex				

$\{0^n 1^n \mid n \geq 0\}$ Überabzählbar unendlich

Beispiel 1: Wortlänge

$ \epsilon = 0$	$ 01010 _1 = 2$	$ a^3b^5 = 8$
$ 01010 _0 = 3$	$ 01010 _3 = \lceil \frac{1}{7} 1291 \rceil = 7$	$ 71291 = 7 \cdot 4 = 28$

Beispiel 2: Sprache

$$L = \Sigma^* \subset \Sigma^*$$
$$L = \emptyset \subset \Sigma^*$$
, die leere Sprache
$$\Sigma = \{0, 1\}$$
, Sprache aller Binärstrings: $L = \Sigma^*$

$\Sigma = \text{Unicode}$, $J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$

M eine "Maschine", $L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\epsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\epsilon) = L$
- $F = \{L(w) \in Q \mid \epsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}$, $L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

BEWEIS

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

- Annahme: L ist regulär
- Gemäss Pumping Lemma gibt es die Pumping Length N
- Wähle ein Wort $w \in L$ mit $|w| \geq N$
- Aufteilung des Wortes gemäss Pumping Lemma
- Auswirkung des Pumpens
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

- Akzeptierzustand $\neq$ Nichtakzeptierzustand
- Iteration: alle unterscheidbaren Paare suchen
- Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5

Beobachtungen 1

- $|w^n| = n \cdot |w|$
- Zu jeder Maschine M gibt es eine Sprache $L(M)$

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition

Tabellenform

Zustandsdiagramm von A

$A = (\Sigma, Q, \delta, q_0, F)$

- Zustände: $Q = \{q_0, q_1, \dots, q_n\}$
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = 0^N 1^N$
- Unterteilung $w = xyz$

TODO: reduce confusion

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

Akzeptieren

$A = (\Sigma, Q, \delta, q_0, F)$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ

Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine Wahl von Übergängen gibt, derart, dass das Wort

Satz

Ein NEA A kann in einen DEA A' umgewandelt werden mit

- $Q' = P(Q)$
- $\Sigma' = \Sigma$
- δ' wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(Q) \mid F \cap M \neq \emptyset\}$

Implementation

Thompson-NEA:

- Zustand $M \subset Q$ realisiert durch Markierung der Zustände in M
- δ' realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

Alle Sprachen $P(\Sigma^*)$

Kontextfrei

Regulär

CFG

PDA

DEA

NEA

regex

$\{0^n 1^n \mid n \geq 0\}$ Überabzählbar unendlich

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots(\delta(q, a_1), a_2), \dots, a_n)$$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

Algorithmus

- $q \in F, q' \notin F$
- Unterscheidbar nach Übergang
- Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

$\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

Transformation NEA $\rightarrow$ DEA

Gegeben $\delta: Q \times \Sigma \rightarrow P(Q)$ eines NEA. Übergangsfunktion für Mengen $M \subset Q$

$$\delta': P(Q) \times \Sigma \rightarrow P(Q): (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$$

Satz

Ein NEA A kann in einen DEA A' umgewandelt werden mit

- $Q' = P(Q)$
- $\Sigma' = \Sigma$
- δ' wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(Q) \mid F \cap M \neq \emptyset\}$

Implementation

Thompson-NEA:

- Zustand $M \subset Q$ realisiert durch Markierung der Zustände in M
- δ' realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

NEA _ε	
Begriff	Bedeutung
ε-Übergang	Kann ohne Verarbeitung eines Zeichens genommen werden. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$
NEA _ε	NEA mit ε-Übergängen

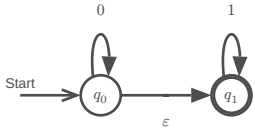
Einen NEA_ε kann man immer in einen NEA umwandeln.
Beweis:

- $$L = \{0^k 1^l \mid k, l \geq 0\}$$
- 1) $E(q)$ = Menge der von q aus mit ε -Übergängen erreichbaren Zustände
- 2) $E(M) = \bigcup_{q \in M} E(q)$
- 3) δ ersetzen durch δ :
 $Q \times (\Sigma \cap \{\varepsilon\}) \rightarrow P(Q) :$
 $(q, a) \mapsto E(\delta(q, a))$

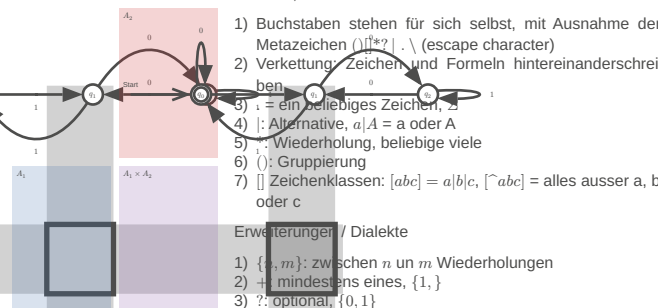
MENGENOPERATIONEN

Lassen reguläre Sprachen regulär sein.

Produktautomat: $L_i = L(A_i)$



$\{w \in \Sigma^* \mid w \text{ ist eine durch drei teilbare Binärzahl}\}$



Reguläre Sprachen sind kontextfrei

1) Ableitung eines Wortes $w \in L(G)$ ist immer in $2|w| - 1$ Regelnanwendungen möglich. Beweis:

- $|w| - 1$ Regeln der Form $A \rightarrow BC$ um aus S ein Wort aus $|w|$ Variablen zu erzeugen
- $|w|$ Regeln der Form $A \rightarrow a$ um das Wort w zu erzeugen
- $\Rightarrow$ insgesamt $2|w| - 1$ Regelnanwendungen

2) Deterministischer Frage: Algorithmen mit Laufzeit $O(n^3)$

2) ε -Regel

$S_0 \rightarrow S$	$S_0 \rightarrow S$
$S \rightarrow ASA \mid aB \mid a$	$S \rightarrow ASA \mid \textcolor{red}{SA} \mid \textcolor{red}{AS} \mid aB \mid a$
$A \rightarrow B \mid \varepsilon \mid S$	$\rightsquigarrow A \rightarrow B \mid S$
$B \rightarrow b$	$B \rightarrow b$

3) Unit-Regel

$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid aB \mid ASA \mid SA \mid AS \mid aB \mid a$$
$$B \rightarrow b \qquad B \rightarrow b$$

4) Komplexe Regeln

$$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$
$$S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$
$$A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$$
$$A_1 \rightarrow SA$$
$$U \rightarrow a$$
$$B \rightarrow b$$

PARSING

COCKE-YOUNGER-KASAMI ALGORITHMUS (CYK)

Gegeben

- 1) Grammatik $G = (V, \Sigma, R, S)$
- 2) Variable $A \in V$
- 3) Wort $w \in \Sigma^*$

Frage

Ist w ableitbar? Formell geschrieben: $A \Rightarrow^* w$

- Spezialfall $w = \varepsilon$: $A \Rightarrow \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$ (Epsilonregel, $A = S$)
- Spezialfall $|w| = 1$: $A \Rightarrow^* w \Leftrightarrow A \rightarrow w \in R$ (Terminalsymbolregel)
- Fall $|w| > 1$:

$$A \Rightarrow^* w \Rightarrow \exists \begin{cases} A \rightarrow BC & \in R \\ w = w_1 w_2 & w_i \in \Sigma^* \end{cases} \text{ mit } \begin{cases} B \Rightarrow^* w_1 \\ C \Rightarrow^* w_2 \end{cases}$$

ABLEITUNGSDREIECK

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle f\u00fcllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gef\u00fcllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

Parse des Wortes $()()()$

$$\rightarrow AB \mid CD \mid CU \mid SSB \mid 1$$
$$\rightarrow SD \qquad C \rightarrow [$$
$$\rightarrow ($$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .

TODO:

rekursion vs iteration (buch 149)

BACKUS-NAUR-FORM (BNF)

Spezifikation der Regeln in maschinen-lesbarer Form:

- Variablen: **<variablen-name>**
- Einzelne Zeichen: A
- Zeichenketten: 'BEISPIEL'
- Regeln: **<variablen-name> ::= Ausdruck**

- Ausdr\u00fccke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch |

Beispiel 10: BNF f\u00fcr Expression-Term-Factor Grammatik

Ausgangsgrammatik

expression $\rightarrow$ expression + term | term

term $\rightarrow$ term * factor | factor

factor $\rightarrow$ (expression) | N

$N \rightarrow NZ \mid Z$

$Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Backus-Naur-Form

<expression> ::= <expression> + <term> | <term>

<term> ::= <term> * <factor> | <factor>

<factor> ::= (<expression>) | <number>

<number> ::= <number> <digit> | <digit>

<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

EXTENDED BACKUS-NAUR-FORM (EBNF)

Definition	Achtung!
- Literale in Anfuhrungszeichen oder Apostroph	- Keine Operatorpriorisierung
- statt ::=	- Zweideutigkeit!
- Variablen ohne < und >, d\u00fcrfen Leerzeichen enthalten	- Keine eindeutige Evaluation!
- Komma f\u00fcr Verkettung	
- Regel-Endzeichen;	
- Kommentare (* ... *)	
- Optionale Wiederholung { ... }	
- Option [...]	
- Gruppierung (...)	
- Ausnahme: -	

Beispiel 11: EBNF f\u00fcr Expression-Term-Factor Grammatik

expression = expression "+" term | term ;

term = term "*" factor | factor ;

factor = "(", expression, ")" | number ;

number = "digit", { digit } ;

digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

RECHTSREKURSION

Expression-Term-Factor-Grammatik verwendet Links-Rekursion -> korrekte Auswertung von links nach rechts. Parsetree wertet aber Ausdr\u00fccke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

expression $\rightarrow$ term expression'

expression' $\rightarrow$ + term expression' | ε

term $\rightarrow$ factor term'

term' $\rightarrow$ * factor term' | ε

factor $\rightarrow$ (expression) | N

$N \rightarrow NZ \mid Z$

$Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

TODO:

grid -> table

STACK

Unendlich grosser Speicher

- Immer nur das oberste Element sichtbar
- Beliebig tief

STACKAUTOMAT / PUSHDOWN AUTOMATON (PDA)

Kann Sprachen akzeptieren, die nicht regul\u00e4r sind, ist aber nicht deterministisch. **Beachte:** $\Gamma \neq \Sigma$ m\u00f6glich

Definition

Stackautomat $(Q, \Sigma, \Gamma, \delta, q_0, F)$

$P =$ **Überg\u00e4nge**

$a, b \rightarrow c$

a vom Input

b vom Stack entfernen (Bedingung)

c auf den Stack

1) Q : Zust\u00e4nde

2) Σ : Eingabe-Alphabet

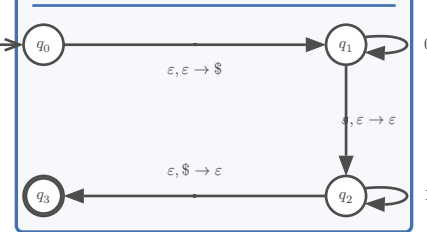
3) Γ : Stack-Alphabet

4) δ : $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow P(Q \times \Gamma_\varepsilon)$

5) $q_0 \in Q$: Startzustand

6) $F \subset Q$: Akzeptierzust\u00e4nde

Beispiel 12: Stackautomat f\u00fcr die Sprache $\{0^n 1^n \mid n \geq 0\}$



TODO:

Book 167-168, shift/reduce

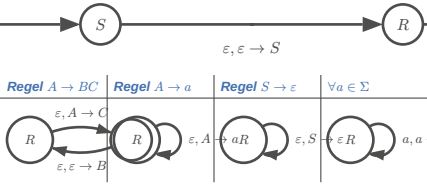
TODO:

diagrams? (slides 8/12)

ABLEITUNG AUS CNF

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P , der L akzeptiert, $L = L(P)$.

Grundger\u00fcst



PDA -> CFG

Variablen

W\u00f6rter beschreiben Pfade durch den Automaten: Variable A_{pq} = W\u00f6rter, die von p nach q f\u00fchren mit leerem Stack

Regeln

Regeln beschreiben, wie sich Wege zerlegen lassen: $A_{pq} \rightarrow A_{pr} A_{rq}$ = Wege von p nach q k\u00f6nnen verstanden werden als Wege von p nach r und von dort nach q

Stackautomat standardisieren

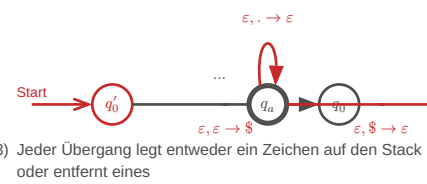
TODO:

better diagrams, book 181

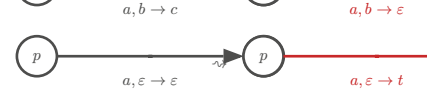
- 1) Nur ein Akzeptierzustand: Neuen Akzeptierzustand q_a und \u00dcberg\u00e4nge von allen vorherigen Endzust\u00e4nden zu q_a erstellen.



- 2) Stack leeren: Mit einem Element \$ erweitern, sodass garantiert werden kann, dass der Stack am schluss leer ist.



- 3) Jeder \u00dcbergang legt entweder ein Zeichen auf den Stack oder entfernt eines



TODO:

Book 182,183

Beispiel 13

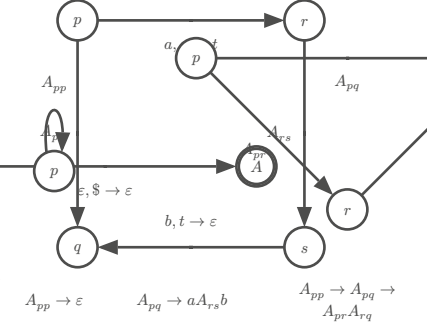
TODO:

visualization

Grammatik ablesen

Ausgangspunkt: standardisierter PDA mit Startzustand q_0 und $F = \{q_a\}$.

- 1) Startvariable: A_{q_0, q_a}
- 2) Regeln:



Beispiel 14

TODO:

NICHT KONTEXTFREIE SPRACHEN

PUMPING LEMMA

Pumping Lemma f\u00fcr CFL

Ist L eine CFL, dann gibt es eine Zahl N , die Pumping Length, derart, dass jedes Wort $w \in L$ mit $|w| \geq N$ zerlegt werden kann in f\u00fcnf Teile $w = uvxyz$ derart, dass

- 1) $|vy| > 0$
- 2) $|vxy| \leq N$
- 3) $uv^k xy^k z \in L \forall k \in \mathbb{N}$

Mit dem Pumping Lemma kann man beweisen, dass eine Sprache **nicht** kontextfrei ist.

Beispiel 15: $\{a^n b^n c^n \mid n \geq 0\}$

1) Annahme: L kontextfrei

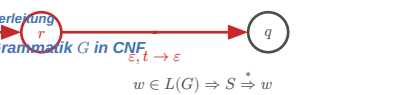
2) Pumping length N

3) Wort: $w = a^N b^N c^N$

4) Zerlegungen:

5) Beim Pumpen nimmt die Anzahl der a und b zu, nicht aber die Anzahl der c
 $\Rightarrow uv^k xy^k z \notin L \forall k \neq 1$

6) Widerspruch: L nicht kontextfrei



Wiederverwendete Variable

$|w| \geq N$ gross genug $\Rightarrow$ Variablen werden im Parse Tree wiederverwendet Die «unterste» wiederverwendete Variable A erzeugt zwei W\u00f6rter:

$$A \Rightarrow^* vxy$$

$$A \Rightarrow^* x$$

Pumpen

$A \Rightarrow^* vxy$ anstelle des «untersten» $A \Rightarrow^* x$ verwenden

TODO:

diagram

UNENDLICH

Begriff	Bedeutung
Endlich	Eine Menge A heisst endlich , wenn jede Injektion $A \rightarrow A$ auch eine Bijektion ist
Abz\u00e4hlar unendlich	Eine Menge A heisst abz\u00e4hlar unendlich , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt, also $A \approx \mathbb{N}$. Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}, \Sigma^*$, Menge aller DEAs/NEAs/PDAs/CFGs
\u00dcberabz\u00e4hlar unendlich	Eine Menge A heisst \u00fcberabz\u00e4hlar unendlich , wenn sie nicht abz\u00e4hlar unendlich ist. Bsp: $\mathbb{C}, P(\mathbb{N})$, Menge aller Sprachen $P(\Sigma^*)$
Gleich m\u00e4chtig	Mengen A und B heissen gleich m\u00e4chtig , $A \approx B$, wenn es eine Bijektion $A \rightarrow B$ gibt
Unendlich	Eine Menge A heisst unendlich , wenn sie gleich m\u00e4chtig wie eine Teilmenge ist

A, B, A_k abz\u00e4hlar unendlich, $k \in \mathbb{N}$:

- 1) $A \cup B, \bigcup_k A_k$ abz\u00e4hlar unendlich
- 2) $A \times B$ abz\u00e4hlar unendlich
- 3) Abz\u00e4hlar unendlich: $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$
- 4) $P(A)$ \u00fcberabz\u00e4hlar unendlich
- 5) $\mathbb{R}$ \u00fcberabz\u00e4hlar unendlich

TURING-MASCHINE (TM)

Mehrere Stacks (Speicherzellen) = RAM (Band).
Jeder DEA oder NEA kann damit emuliert werden.

TODO:

diagram (slides 3)

- 1) Der Speicher ist unbegrenzt
- 2) In jeder Zelle genau ein Zeichen
- 3) Es ist immer nur eine Speicherzelle einsehbar
- 4) Der Inhalt der aktuellen Zelle kann beliebig ver\u00e4ndert werden
- 5) Bewegung immer nur eine Zelle nach links oder rechts
- 6) Kein weiterer Speicher (nur die Zust\u00e4nde eines endlichen Automaten und das eine Zeichen)

Begriff	Bedeutung
Record	Speichereinheit fester grösse
Bandalphabet	Alphabet Γ , welches aus Speicherzellen (Stacks) besteht
Schreib-/Lesekopf	Aktuelle Schreib-/Leseposition

Definition

(deterministische) Turing-Maschine
 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

- Q : Zustände
- Σ : Alphabet
- Γ : Bandalphabet, $\cup \in \Gamma \setminus \Sigma$
- δ : $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $q_0 \in Q$: Startzustand
- $q_{\text{accept}} \in Q$: Akzeptierzustand
- $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

ARBEITSWEISE

TODO:
diagram (slides 8/3)

Programmablauf

- Input-Wort w auf Band
- Schreib- / Lesekopf positioniert auf 1. Zeichen
- Maschine starten, $t(w)$ Einzelschritte ausführen
- Maschine hält in q_{accept} oder q_{reject} : Wort w akzeptiert / Verworfen

Laufzeit: $t(w), t(n) = \max\{t(w) \mid |w| \leq n\}$

Beispiel 16

TODO:
Zustandsdiagramm + Turing machine diagram 0^n 1^n

TODO:
Protokollierung

TODO:
Mehrspurmaschine

TODO:
Mehrbandmaschine

NICHTDETERMINISTISCHE TM

Übergangsfunktion Simulation auf Standard-TM

Bei jedem Übergang maximal N verschiedene Möglichkeiten:
 $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$
 $\Rightarrow$ Mehrere mögliche Berechnungswege

Verwende 3 Bänder:

- 1) Arbeitsband
- 2) Kopie von w
- 3) Liste aller Folgen von Wahlmöglichkeiten

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

Simulationsidee

Alle Berechnungswege durchführen, maximal $N^{t(n)}$

Laufzeit: $O(N^{t(n)}) = 2^{O(t(n))}$

TODO:
Vergleiche

TODO:
Aufzähler

ENTSCHEIDER UND ENTSCHEIDBARE SPRACHEN

Deterministisch: Eine TM M heisst **Entscheider**, wenn sie auf jedem Input a anhält.

Nichtdeterministisch: Eine nichtdeterministische TM M heisst **Entscheider**, wenn jede Berechnungsgeschichte terminiert.

Turing-erkennbare Sprache: L heisst **Turing-erkennbar**, wenn es eine TM M gibt mit $L = L(M)$.

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.